Agenda

1. Fundamentals and core principles
    - What is a programming language?
    - What is Python?
    - Variables
    - Assignment to variables using `=` (assignment operator)
    - Displaying values with `print`
    - Comparison operators, e.g., `==`
    - Conditional evaluation with `if` / `elif` /'else`
    - Numbers (`int` and `float`)
    - Text (`str` , aka "string")
    - Methods, especially string methods
2. Loops, lists, and tuples
    - Repeating actions with loops
        - `for` and `while`
        - Indexes
        - Controlling your loops with `break` and `continue`
        - Infinite loops
    - Lists (another data type)
        - creating lists
        - Retrieving from them
        - List are mutable -- they can be changed!
    - Turning strings into lists, and back using `str.split` and `str.join`
    - Tuples and tuple unpacking
3. Dictionaries and files
    - What are dicts?
    - Reading/writing dicts
    - Iterating over them
    - Files -- reading from them and (a little) writing to them
4. Functions -- defining our own verbs in Python
    - Defining our own functions
    - Arguments and parameters
    - Return values
    - Local vs. global variables
5. Modules and packages
    - Using modules with `import`
    - Writing our own modules
    - The Python standard library
    - PyPI, the third-party library

- Where to next?

How and where can you write Python code?

If you're going to write Python, you're going to need two things: (a) a copy of the Python language and (b) an editor that's familiar with Python, and lets you create files -- and probably run them.

Traditionally, you would do this by downloading Python from python.org, and then installing it, and then installing an editor -- such as PyCharm or Visual Studio Code.

There is, however, an alternative called Jupyter that has become massively popular. I'm typing into Jupyter now! It's known as a "notebook," and it's particularly popular in the data community. You can install Jupyter on your own computer, once you've installed Python.

Jupyter gives us a highly interactive enviroment in which we can write code, see the reuslts of our code right away, and iterate quickly.

But... installing it is a bear if you're new. Installing Python is easier than used to be the case, but it's also annoying.

Instead, use Jupyter Lite

This is a 100% browser-based version of Jupyter. You can go to it at

https://jupyter.org/try-jupyter/lab/

Intro to using Jupyter

Jupyter works on a system of "cells." Right now, I'm typing into a cell. Each cell at a given moment, can be in one of two modes (when I'm typing into it):

- Edit mode. When I'm in edit mode, typing enters characters, like this. I can get into edit mode by clicking in the cell or by pressing Enter.
- Command mode. When I'm in command mode, typing a character doesn't make it visible. Rather, it tells Jupyter to do something. I can get into command mode by clicking to the left of the cell or by pressing ESC.

A few commands:

- `c` -- copies the cell
- `x` -- cuts the cell

- `v` -- pastes the most recently copied/cut cell in the notebook
- `a` -- add a new cell *above*
- `b` -- add a new cell *below*
- `m` -- put the cell into Markdown (documentation) mode
- `y` -- put the cell into coding (Python) mode

When I'm done editing a cell in edit mode, I can press shift+ENTER together, and that "executes" the cell, meaning that the code runs.

What is a programming language?

Back when computers were invented, they were collections of vacuum tubes, wires, etc. If you wanted to solve a problem, you built a computer to do so. If you wanted to solve a different problem, you built a separate computer to do so. This was clearly very inefficient.

At some point, people realized that it would be better to build a computer a single time, and solve many different problems. This was the birth of the *programmable computer*. You would program it using 1s and 0s. That worked, but was really tedious and annoying.

That led to the creation of high-level programming languages. You would write in something resembling English, and a second program (a language) would translate your code into 1s and 0s.

That is largely how programming languages work today! You write your code, it is translated into the 1s and 0s, and it is executed.

There are many *many* different programming languages out there. Each has its own advantages and disadvantages.

- C -- runs very very fast, because it's so similar to the 1s and 0s. However, it's hard for people to work with, and the coders need to keep track of how much memory has been used.
- Rust -- a very new language, competing with C, trying to make it easier to write code that runs fast and doesn't require lots of worrying about memory
- Java -- an even higher level language, which now runs super fast, but which took some time to take off and become popular. It uses a style of programming known as "objects," which we won't really be talking about in this course.
- Python -- a very high level language, meaning that it looks roughly like English in many places. The ideaw of Python is to make it easy for someone to learn to program and then to actually code.

Python

It was created and released about 33 years ago, by Guido van Rossum. The goal was to create a programming language that would be easy for people to learn and use, but not limit them.

30+ years later, Python is *amazingly* popular:

- # 1 language for data -- data science, machine learning, etc.

- automated testing
- web applications
- APIs (servers and clients)
- Internet of things
- Education

Let's write some Python!

- I have a notebook open on my computer
- I'll go into a new cell, and make sure the cell is for Python + coding (and note Markdown/documentation)
- I'll write some Python
- I'll execute it with shift+Enter.

In [1]:
```
# these are comments (anything after a # mark), ignored by Python completely
# we can invoke the "print" function, which displays something on the screen
# to invoke a function, you must use () after its name
# in the (), you can pass an *argument*.

print('This is amazing! (But what do you think?)')
```

```
This is amazing! (But what do you think?)
```

In this case, we passed a string (text) as an argument (value passed to the function that will be used in the function).

In [2]:
```
print(10)  # can I print numbers?
```

```
10
```

In [3]:
```
# can I print the result of a numeric operation?

print(10 + 2)
```

```
12
```

In [4]:
```
# can I add together text?

print('abcd' + 'wxyz')
```

```
abcdwxyz
```

In [5]:
```
# what happens if I try to add `10` to `'wxyz'`?

print(10 + 'wxyz')
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[5], line 3
      1 # what happens if I try to add `10` to `'wxyz'`?
      2
----> 3 print(10 + 'wxyz')

TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

Variables and assignment

If you have to type each value you want to display, it'll be very hard to maintain this code.

A variable is basically a pronoun in the Python foreign language. It refers to a value that you have already created.

We can asisgn any value to a variable, and then we can ask for the variable's value later on, and it's still there.

In [7]:
```
v = 1234
print(v)
```

```
1234
```

In [8]:
```
print(v + 3)
```

```
1237
```

`=` is not equality!

This one-character symbol doesn't mean the same thing as you learned in school. Here, it's just a way of assigning the value on the right to the variable on the left.

In [9]:
```
x = 10
y = 20

print(x + y)
```

```
30
```

Why no declarations?

In many other langugages, when (or before) I assign to a variable, we must declare the variable to be of a certain type -- number, text, etc.

This doesn't really exist in Python.

The reason you need this in other languages is because they need to know how much memory to allocate. In those languages, a variable is basically an alias for a location in memory. If the location isn't the right size for the data, bad news.

Any variable in Python can refer to any value. So there's no need for that declaration.

Exercise: Simple asisgning and printing

1. Assign a number to `x`. Assign a different number to `y`. Assign their sum to `total`. Print `total`.
2. Assign your name to the variable `first_name`. Assign your city to the variable `city_name`. Print, using `print` and `+`, a greeting to the person and their city.

Note that there is a big difference between `first_name` (a variable, without quotes) and `'first_name'` (a string, with quotes).

In [10]:
```
# 1. Assign a number to `x`. Assign a different number to `y`. Assign their

x = 1234
y = 5678

total = x + y

print(total)
```

```
6912
```

In [12]:
```
# 2. Assign your name to the variable `first_name`. Assign your city to the

first_name = 'Reuven'
city_name = 'Modiin'

print('Hello ' + first_name + ' from ' + city_name)
```

```
Hello Reuven from Modiin
```

A few things to notice

1. In Python, you *can* use capital letters in variable names, but most people don't. So use `first_name`, not `First_name`. THESE ARE DIFFERENT VARIABLES! If you use a name capitalized (e.g., `X`) and you defined it with lowercase, then there won't be a match.

2. This is true for Python's functions, too! If you call `Print` or `PRINT`, neither will work. It has to be lowercase `print`.
3. Variable names (into which we assign) have a few rules and a few conventions:
   - We normally only use lowercase letters
   - If we need multiple words, we use `_` between the words
   - You can use lowercase letters, uppercase letters (but try to avoid them), `_`, and digits.
   - Don't start or end a variable name with `_`
   - Don't start a variable name with a digit.
4. There is a world of difference between `x` (the variable) and `'x'` (the letter x).

Quotes for text

In some programming languages, you use `''` to mark of text, as in `'hello'`. In others, you use double quotes, e.g., `"hello"`. Which is best in Python?

First of all: It doesn't matter at all which you use. Try to be consistent, but Python couldn't care less.

Spaces -- in strings and in variable names

When you assign, there are two parts to the assignment:

- On the left, we have a variable name. No quotes, one word, all lowercase (by convention), and use `_` between "words".
- On the right, we have a value. That value, if it's a text string, must be in quotes, and can contain *ANY* characters you want, without exception.

In [13]:
```
# variable name is city_name
# text value is 'New York'

city_name = 'New York'
```

In [14]:
```
print(city_name)
```

```
New York
```

In [15]:
```
# what if I do this?

city name = 'New York'
```

```
  Cell In[15], line 3
    city name = 'New York'
         ^
SyntaxError: invalid syntax
```

In [16]:
```
# what if I do this?

city_name = New York
```

```
  Cell In[16], line 3
    city_name = New York
                     ^
SyntaxError: invalid syntax
```

In [ ]:
```

```

Next up:

1. How to get information from the user with the `input` function
2. Comparing values
3. Conditional code with `if`

Getting input

It's very nice that I can assign to variables, and I can then print their contents. But ... that gets pretty boring pretty quickly. I'd like my program to handle not just values it knows when it was written ("hard coded") but also values that the user enters.

To get input from the user, we'll use the `input` function.

- You invoke `input`
- In the `()`, you put a text string, whatever you want it to say/ask of the user
- `input` *returns* a value, text, whatever the user entered
- Typically/traditionally, we take the value that `input` returns, and assign it to a variable.

When `input` runs, the program stops and waits for input from the user.

In [19]:
```
# here, I'm calling input
# whatever it returns is assigned to name

name = input('Enter your name: ')

print('Hello, ' + name + '!')
```

```
Hello, ;sdjlasdk;jfsalk;dfjafl;dkasjfl;kajfs;lafja!
```

In [18]:
```
print(name)
```

```
Reuven
```

A warning

The return value from `input` is always (always!) a text string. Which means you will end up with this:

In [20]:
```
n = input('Enter your favorite number: ')

print('Your favorite number, added to itself, is ' + n+n)
```

```
Your favorite number, added to itself, is 1515
```

Comparison operators

So far, we've seen the `+` operator, which allows us to add together two values. The return value from that operator is a new value -- often a new number, but sometimes a new text string.

How about a different comparison operator, this one `==` (yes, double equal signs).

`==` takes two values, and compares them. If they are the same value, then we get back the special value `True`. If not, then we get back the special value `False`.

In [21]:
```
# in Jupyter, and *only* in Jupyter, if the final line of a cell has a value
# no call to `print` is needed.

10 == 10
```

Out[21]: True

In [22]:
```
10 == 5
```

Out[22]: False

In [23]:
```
x = 10
y = 10

x == y
```

Out[23]: True

In [24]:
```
'abc' == 'ABC'
```

Out[24]: False

In [25]:
```
'abc' == 'abc'
```

Out[25]: True

In [26]:
```
'abc ' == 'abc'
```

Out[26]: False

In [27]:
```
10 == '10'  # is the integer 10 equal to the text string 10?
```

Out[27]:
```
False
```

Other comparison operators

Python has a standard set of comparison operators:

- `==` -- equal to
- `!=` -- unequal to (like ≠)
- `<` -- less than
- `<=` -- less than or equals
- `>` -- greater than
- `>=` -- greater than or equals

In [28]:
```
10 < 20
```

Out[28]:
```
True
```

In [30]:
```
# if you use < or > to compare text, it'll look at their alphabetical positi

'hello' < 'goodbye'
```

Out[30]:
```
False
```

Conditional code execution

So far, our programs have been tiny and simple, but they also have another quality: They run from the start to the end. No matter what, all of the code will run.

Rarely do we really want programs to work this way. Often, we want only a part of our code to run. Not all parts are relevant all of the time.

To handle this, we have `if`:

- `if` looks to its right for a `True` or `False` value.
  - If it's `True`, then the accompanying block of code runs.
  - If it's `False`, then that code does not run.
- After `if` you can have an `else` clause
  - This block runs if the condition was `False`.

In [31]:
```
x = 5

if x > 3:
    print('Yes, greater than 3!')
```

```
else:
    print('Smaller than or equal to 3')
```

```
Yes, greater than 3!
```

Look at the above code:

- `if` looks to its right for a condition
- After the condition, we have `:` -- that indicates the following line has an indented block
- The indentation is traditionally 4 spaces per level -- after a short while, you'll forget about how to write indentation; your tools will typically do it for you.
- You can have as many lines as you want in an indented block.
- When you want to finish the block, just backspace back to the previous indentation level, typically 4 spaces
- You can also have an `else` block; when the condition is `True`, the `if` block runs-- and if it's `False`, the `else` block runs.

In [34]:
```
name = input('Enter your name: ')

if name == 'Reuven':
    print('Hello, boss!')
    print('You were really missed from around the office.')
else:
    print('Who are you?')
```

```
Who are you?
```

Exercise: Which word comes first:

1. Ask the user to enter a word, and assign to `first_word`.
2. Ask the user to enter a second word, and assign to `second_word`.
3. We can assume that the words are different and both lowercase. Also, they contain no punctuation or spaces.
4. Tell the user which comes first alphabetically.

In [35]:
```
'hello'  # totally fine
```

Out[35]:
```
'hello'
```

In [36]:
```
"hello"  # also totally fine
```

Out[36]:
```
'hello'
```

In [38]:
```
# when you put something in quotes, you're saying, "I want to see this preci

print('first_word' + 'second_word')
```

```
first_wordsecond_word
```

In [40]:
```
first_word = 'hello'
second_word = 'goodbye'

print(first_word + second_word)
```

```
hellogoodbye
```

In [42]:
```
first_word = input('Enter first word: ')
second_word = input('Enter second word: ')

if first_word < second_word:
    print(first_word + ' comes first.')
else:
    print(second_word + ' comes first.')
```

```
love comes first.
```

What if we enter the same word twice?

Right now, our program doesn't handle that. It's give us the wrong answer.

We could, however, fix it:

In [46]:
```
first_word = input('Enter first word: ')
second_word = input('Enter second word: ')

if first_word < second_word:   # if first_word comes first...
    print(first_word + ' comes first.')
else:
    if first_word > second_word:  # if second_word comes first...
        print(second_word + ' comes first.')
    else:
        print('You entered ' + first_word + ' twice!')
```

```
abc comes first.
```

The better way to do this is to use three clauses:

- `if` - the main comparison
- `elif` -- another comparison, after `if`, with its own block
- `else` -- if nothing else is `True`

In [47]:
```
first_word = input('Enter first word: ')
second_word = input('Enter second word: ')

if first_word < second_word:   # if first_word comes first...
    print(first_word + ' comes first.')
elif first_word > second_word:  # if second_word comes first...
    print(second_word + ' comes first.')
else:
    print('You entered ' + first_word + ' twice!')
```

```
goodbye comes first.
```

In [49]:
```python
# Viraj

first_word = input('Enter a first word: ')
second_word = input('Enter a second word: ');

if first_word > second_word:
    print('First word')
else:
    print('Second word')
```

```
First word
```

In [ ]:
```python
first_word = input('Enter first word: ')
second_word = input('Enter second word: ')

if (first_word < second_word):   # if first_word comes first...
    print(first_word + ' comes first.')
elif (first_word > second_word):  # if second_word comes first...
    print(second_word + ' comes first.')
else:
    print('You entered ' + first_word + ' twice!')
```

Many years ago, someone said to me: Computers don't do what you want them to do. They do what you tell them to do.

In [50]:
```python
# Paul

first_word=input('first_word: ')
second_word=input('second_word: ')

if first_word < second_word:
    print(first_word)
else:
    print(second_word)
```

```
romeo
```

Let's talk about f-strings

This is one of the best modern additions to Python, "format strings" or "f-strings."

The idea is:

- The string starts with `f`, before the opening quote
- Inside of the string, everything is totally normal *except* you can have `{}`
- Inside of `{}`, you can put a Python expression, meaning something that has a value:
  - a variable name, `{x}`
  - an expression, `{x + y}`

This not only makes our strings easier to write and read, but it also avoids a huge problem -- namely, that cannot easily put integers into a string.

In [52]:
```
x = 5
print('My favorite number is ' + x)   # this won't work -- you cannot + a st
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[52], line 2
      1 x = 5
----> 2 print('My favorite number is '+ x)

TypeError: can only concatenate str (not "int") to str
```

In [53]:
```
print(f'My favorite number is {x}')   # now, problem solved!
```

```
My favorite number is 5
```

In [54]:
```
# rewritten exercise solution using f-strings

first_word = input('Enter first word: ')
second_word = input('Enter second word: ')

if first_word < second_word:
    print(f'{first_word} comes before {second_word}')
elif first_word > second_word:
    print(f'{second_word} comes before {first_word}')
else:
    print('You entered ' + first_word + ' twice!')
```

```
love comes before marriage
```

In [55]:
```
x = 5

print(f'x = {x}')  # this is super common
```

```
x = 5
```

In [56]:
```
# there is another way to do it:

print(f'{x=}')   # inside of the {}, I put a Python expression and immediate
```

```
x=5
```

In [57]:
```
x = 5
y = 6

print(f'{x+y=}')  # the expression/value inside of {} is x+y
```

```
x+y=11
```

In [58]:
```
f'abcd'   # this is a format string, aka f-string -- because of an earlier i
```

Out[58]:
```
'abcd'
```

Next up

1. Combining conditions with `and`, `or`, and even `not`
2. Talk about numbers (`int` and `float`)
3. Start to talk about strings

Combining conditions

We've seen how we can use `if` with one condition. What if I want to check that two conditions are `True`?

One possibility is to have an `if` inside of another `if`. That is: The block for the outer `if` contains another one. This gets hard to read/understand.

However, we can instead use `and` to combine two conditions.

- On one side of `and`, we put the first condition
- On the second side, we put the second condition
- The total returns `True` if both of these conditions are `True`.
- Notice that we aren't putting values on either side of `and`, but rather conditions

In [59]:
```
x = 10
y = 20


# exaggerated spaces, though they work
#         True       and      True
if     x == 10      and      y == 20:
    print('Yes! Both are what you want')
```

```
Yes! Both are what you want
```

In [62]:
```
x = 10
y = 20


# exaggerated spaces, though they work
#         True       and      False
if     x == 10      and      y == 998877:
    print('Yes! Both are what you want')
else:
    print('Bad news, neither is what you want.')
```

```
Bad news, neither is what you want.
```

Another way to combine: `or`

The `or` condition basically says: If either one of these two is `True`, then so am I.

In [64]:
```
#         True       or      False
if     x == 10      or      y == 2345:
    print('Yes! at least one is what you want')
```

```
Yes! at least one is what you want
```

Another logical operator: `not`

The `not` operator goes in front of a `True` or `False` value, and returns the opposite.

In [65]:
```
x = 5

if x == 5:
    print('Yes! It is 5!')
```

```
Yes! It is 5!
```

In [67]:
```
x = 6

if not x == 5:   # really, you should never do this -- just use != instead c
    print('No! It is not 5!')
```

```
No! It is not 5!
```

In [ ]:
```

```

Exercise: Name and company

1. You're going to ask the user to enter their first name and their company's name. You're going to assign each to a variable.
2. Then give one of the following answers:
   - If the name and company match yours, then say it must be you
   - If the name matches but the company doesn't, then compliment their name and be snarky about their company.
   - If the name doesn't match, but the company does, say hello to your colleague
   - If neither matches, say something snarky.

In [70]:
```
my_name = 'Reuven'
my_company = 'LernerPython'

name = input('Enter your name: ')
company = input('Enter your name: ')

if name == my_name and company == my_company:
    print('You must be me!')
elif name == my_name:    # we know that the company can't be the same!
    print('Great name, horrible company!')
```

```
elif company == my_company:
    print('Hello, my colleague!')
else:
    print('Ugh, I dislike your name and your company.')
```

```
Great name, horrible company!
```

Data structures

We know that at the end of the day, everything in a computer is 1s and 0s. But we can collect a bunch of them into a higher-level data structure, much as we can collect molecules into objects in our world. Different types of data behave differently -- numbers do different things from text strings, which do different things from lists and dictionaries.

Integer

Integers (`int`) are whole numbers. Meaning that they from very small (negative) numbers to very large (positive) numbers. Unlike some languages, Python's ints aren't limited. You can get as negative as you want, or as postive as you want.

In [71]:
```
x = 10

type(x)  # what kind of value is x?
```

Out[71]:
```
int
```

In [72]:
```
x = 10
y = 3
```

In [73]:
```
x + y  # addition
```

Out[73]:
```
13
```

In [74]:
```
x - y   # subtraction
```

Out[74]:
```
7
```

In [75]:
```
x * y # multiplication
```

Out[75]:
```
30
```

In [77]:
```
x / y   # truediv -- returns a "float" -- a number with a fractional part at
```

Out[77]:
```
3.3333333333333335
```

In [78]:
```
x // y   # floordiv -- rounds the number to the lower number before it.
```

Out[78]: 3

In [79]:
```
x ** y   # exponentiation
```

Out[79]: 1000

In [80]:
```
x % y   # modulo (really, remainder from division)
```

Out[80]: 1

In [82]:
```
x = 10

x = x + 1    # we're saying: first calculate x+1, and assign to x.

x
```

Out[82]: 11

In [83]:
```
# we do have a shortcut, though:

x += 1  # this calculates x+1, and then assigns it back to x
```

In [86]:
```
x = 10
y = 20

if x == 10 and y != 20:
    print('first')

# if so, then what's wrong with this:

if x == 10 and not y == 20:
    print('second')
```

In [87]:
```
# I've ignored a problem so far

x = 10
y = '20'   # this is a string, not an integer!

print(x+y)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[87], line 6
      2
      3 x = 10
      4 y = '20'   # this is a string, not an integer!
      5
----> 6 print(x+y)

TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

In [88]:
```
# the real solution is to create a new integer based on our string (y)
# we can do that by invoking the "int" function
```

```
print(x+int(y))   # notice -- this doesn't do anything to change y into an i
```

```
30
```

In [89]:
```
# Instead, you could do this:

y = int(y)  # assign back to y the int value, and now y is permanently an in
```

You need to use int to turn any input (normally a string) into an integer, in many cases.

Exercise: Guessing game

1. Assign an integer to `secret`.
2. Print the current value of `secret` using an f-string.
3. Ask the user to guess, and assign to `guess`.
4. Print one of three things:
    - You got it!
    - Too low!
    - Too high!

The user only gets one chance.

In [91]:
```
secret = 72
print(f'secret = {secret}')

guess = input('Enter your guess: ')

if guess == secret:       # I compare a string and int -- they aren't ==!
    print('You got it!')
elif guess < secret:      # I compare a string and an int... here, it breaks
    print('Too low!')
else:
    print('Too high!')
```

```
secret = 72
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[91], line 8
      4 guess = input('Enter your guess: ')
      5
      6 if guess == secret:
      7     print('You got it!')
----> 8 elif guess < secret:
      9     print('Too low!')
     10 else:
     11     print('Too high!')

TypeError: '<' not supported between instances of 'str' and 'int'
```

In [92]:
```
secret = 72
print(f'secret = {secret}')

guess = input('Enter your guess: ')    # or guess = int(input('Enter your gu
guess = int(guess)

if guess == secret:
    print('You got it!')
elif guess < secret:
    print('Too low!')
else:
    print('Too high!')
```

```
secret = 72
You got it!
```

In [94]:
```
# Chris

secret = '5'    # the string 5!!
x = input('What is my lucky number?')

if x == secret:
    print(f'{secret} Is the right answer!')
elif x <= secret:
    print(f'{secret} Too high!')
else:
    print(f'{secret} Too Low!')
```

```
5 Too high!
```

In [95]:
```
'40' <= secret
```

Out[95]: True

In [96]:
```
# if you keep them as strings, they'll be compared as strings
# strings are compared like dictionary words - -check the first letter, ther
```

In [98]:
```
# Chris

secret = 5
x = input('What is my lucky number?')
x = int(x)

if x == secret:
    print(f'{secret} Is the right answer!')
elif x > secret:
    print(f'{secret} Too high!')
else:
    print(f'{secret} Too Low!')
```

```
5 Too high!
```

In [100…
```
# Alexandria

secret = 2
print (f'The current value of secret is {secret}')
```

```
guess = input('Hello user, please guess the current value of secret: ')

if int(guess) == secret:
    print ('You got it!')
elif int(guess) <= secret:
    print(str('Too low'))
elif int(guess) >= secret:
    print(str('Too high'))
```

```
The current value of secret is 2
Too high
```

Floats

So far, we've talked about integers, which are whole numbers. You sometimes need fractional numbers. These are called "floats."

Every float as a decimal point, which means it'll often have .0.

In [101…
```
x = 123.45
type(x)
```

Out[101…
```
float
```

Next up

1. Strings
2. String methods

Strings

A "string" (known as `str` in Python) is used for any text at all in our programming. It can be empty (`''`), or it can be many hundreds of gigabytes, if your computer has that kind of memory. It's still considered a "string."

As we've seen, you can create a string with `''` (single quotes) or `""` (double quotes). There is no difference from Python's perspective. You can use the different quotes in order to have quote characters in your string. For example:

```
s = "He's very nice."   # avoid trouble with ' in the string
by using ""
```

In [102…
```
s = 'abcdefghijklmnopqrstuvwxyz'

len(s)  # this function, len, returns an integer – –the length of the string
```

Out[102… 26

In [103…
```
# what if I want to retrieve the first character?
# I can use [], putting the index of the character I want inside

# what index does the first character have? The answer is: 0
# Python starts indexing at 0

s[0]
```

Out[103… 'a'

In [104…
```
s[1]
```

Out[104… 'b'

In [105…
```
# I can use variables, too!

i = 3
s[i]  # what is s at index i?
```

Out[105… 'd'

In [106…
```
# What if I want the final character?

s[26]
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[106], line 3
      1 # What if I want the final character?
      2
----> 3 s[26]

IndexError: string index out of range
```

In [107…
```
s[ len(s)-1 ]
```

Out[107… 'z'

In [108…
```
# there's another way to do this -- just use a negative index!

s[-1]  # this returns the final character
```

Out[108… 'z'

In [109…
```
s[-2]
```

Out[109… 'y'

In [110…
```
s[-3]
```

Out[110… 'x'

In [111…
```
# slice!

s[10:20]  # this returns a new string, based on s, from index 10 up to and r
```

Out[111…
```
'klmnopqrst'
```

In [112…
```
s[:20]   # from the start, to index 20
```

Out[112…
```
'abcdefghijklmnopqrst'
```

In [114…
```
s[10:]  # from index 10 through the end
```

Out[114…
```
'klmnopqrstuvwxyz'
```

In [115…
```
s = 'abc def ghi'
len(s)
```

Out[115…
```
11
```

Exercise: Get a character

1. Ask the user to enter a string, and assign to `text`.
2. Ask the user to enter an index, and assign to `i`, making sure to turn it into an integer.
3. If `i < 0`, give an error message.
4. If `i` is too big, give an error message.
5. Otherwise, print `text`, `i`, and the character at that point.

In [117…
```
text = input('Enter text: ')

i = input('Enter index: ')
i = int(i)

if i < 0:
    print('Error; too small')
elif i >= len(text):
    print('Error, too big')
else:
    print(f'{text=}, {i=}, {text[i]}')
```

```
text='hello out there', i=4, o
```

Searching in a string

If we want to know if one string is in another, we can use `in`:

```
'a' in 'abcde'   # True
'b' in 'hello'   # False
```

```
'ac' in 'access' # True
'ae' in 'access' # False
```

Exercise: Pig Latin translator

1. Ask the user to enter a single English word in lowercase. (No punctuation, either).
2. If the first letter is a vowel (a, e, i, o. u), add `way` to the word and print it.
3. If the first letter is *not* a vowel, then move the first letter to the end and add 'ay'.

In [121…
```
word = input('Enter word: ')

if word[0] == 'a' or word[0] == 'e' or word[0] == 'i' or word[0] == 'o' or w
    print(word + 'way')
else:
    print(word[1:] + word[0] + 'ay')
```

```
rocodilecay
```

In [122…
```
# what's with line 3?

word = input('Enter word: ')

if word[0] in 'aeiou':
    print(word + 'way')
else:
    print(word[1:] + word[0] + 'ay')
```

```
appleway
```

In [124…
```
word = input('Enter word: ')
i = input('Enter index: ')
i = int(i)

if i < 0:
    print('Error; too small')
elif i >= len(text):
    print('Error, too big')
else:
    print(f'{text=}, {i=}, {text[i]}')
```

```
text='hello out there', i=3, l
```

Methods

So far, we've discussed and used a number of functions. But some functions don't float around. Rather, they attached to one particular object. These are known as "methods." Methods work the same as functions, but they look a little different.

A method always looks like

```
x.MODNAME
x.MODNAME(10, 20, 30)
```

In [126…
```
name = input('Enter your name: ')

print(f'Hello, {name}!')
```

```
Hello,                Reuven                    !
```

Method 1: `str.strip` -- this returns a new string, just like the original, but without whitespace on either side.

In [127…
```
name = input('Enter your name: ')
name = name.strip()

print(f'Hello, {name}!')
```

```
Hello, !
```

In [128…
```
name = input('Enter your name: ').strip()

print(f'Hello, {name}!')
```

```
Hello, Reuven!
```

Some other string methods

- `str.lower` -- returns a lowercase version of the string
- `str.upper` -- returns an UPPERCASE version of the string
- `str.isdecimal` -- returns True if the string only contains digits 0-9.

In [131…
```
s = '12345 '

s.isdecimal()
```

Out[131…
```
False
```

Exercise: Improve the guessing game

1. Take the guessing game, modify it such that if the user enters a non-integer, they get a warning.
2. Otherwise it should remain the same.

In [133…
```
# word is a string, what the user entered
# [0] is an index retrieval mechanism, just like tuples and lists (as we'll
# really, word[0] is just the first letter

# we use "in" to check if word[0] is in the string 'aeiou'.
```

```
if word[0] in 'aeiou':
    print('Vowel')
```

In [139…

```
secret = 72
print(f'secret = {secret}')

guess = input('Enter your guess: ')

if guess.isdecimal():  # this is a string method, checking if guess contains
    guess = int(guess)

    if guess == secret:
        print('You got it!')
    elif guess < secret:
        print('Too low!')
    else:
        print('Too high!')
else:
    print('Try a numeric value next time!')
```

```
secret = 72
Try a numeric value next time!
```

In [ ]: